UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

CS3014 - Estructuras de Datos Avanzadas

Enteros I
Semana 11

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

21 de octubre de 2026

Objetivos de la sesión



Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Precisar los modelos de cómputo en los que trabajan las estructuras de enteros
- Plantear el problema del predecesor y entender por qué $\Omega(\lg n)$ no es una barrera cuando las llaves son enteros

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Precisar los modelos de cómputo en los que trabajan las estructuras de enteros
- Plantear el problema del predecesor y entender por qué $\Omega(\lg n)$ no es una barrera cuando las llaves son enteros
- Construir la estructura de van Emde Boas: Sucesor, Predecesor, Insertar y Eliminar en $O(\lg \lg u)$
- Usarla para ordenar y relacionar ordenamiento con colas de prioridad

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Una tabla de rutas

Un router guarda n prefijos de direcciones IP de 32 bits y, para cada paquete, busca el mayor prefijo menor o igual que su dirección: un **predecesor**.

¿Los árboles de búsqueda son lo mejor que podemos hacer?

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Una tabla de rutas

Un router guarda n prefijos de direcciones IP de 32 bits y, para cada paquete, busca el mayor prefijo menor o igual que su dirección: un **predecesor**.

¿Los árboles de búsqueda son lo mejor que podemos hacer?

Solo si lo único que sabemos hacer con una llave es compararla

$\Omega(\lg n)$ es una cota del **modelo de comparación**. Pero una dirección IP es un entero de 32 bits: se puede sumar, dividir en mitades, usar como índice de un arreglo. ¿Qué cambia entonces?

Modelos de cómputo

Palabras de w bits

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El universo

Las llaves son enteros de w bits: $U = \{0, 1, \ldots, u-1\}$ con $u = 2^w$. Una **palabra** es una celda de memoria de w bits.

Palabras de w bits

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El universo

Las llaves son enteros de w bits: $U = \{0, 1, \ldots, u-1\}$ con $u = 2^w$. Una **palabra** es una celda de memoria de w bits.

Ejemplos

Direcciones IPv4: $w = 32$, $u \approx 4 \cdot 10^9$. Enteros de máquina: $w = 64$, $u \approx 1{,}8 \cdot 10^{19}$.

Con n llaves y S celdas de memoria, ¿cuánto tiene que valer w?

Palabras de w bits

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El universo

Las llaves son enteros de w bits: $U = \{0, 1, \ldots, u-1\}$ con $u = 2^w$. Una **palabra** es una celda de memoria de w bits.

Ejemplos

Direcciones IPv4: $w = 32$, $u \approx 4 \cdot 10^9$. Enteros de máquina: $w = 64$, $u \approx 1{,}8 \cdot 10^{19}$.

Con n llaves y S celdas de memoria, ¿cuánto tiene que valer w?

Al menos $\lg S$: si no, una palabra no podría guardar una dirección de memoria.
Entonces $w \geq \lg n$.

Transdichotomous RAM y word RAM

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Transdichotomous RAM

La memoria es un arreglo de S celdas de w bits, con $w \geq \lg S$. Cada operación toca $O(1)$ celdas y las palabras sirven como punteros: se puede acceder a cualquier celda.

Transdichotomous RAM y word RAM

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Transdichotomous RAM

La memoria es un arreglo de S celdas de w bits, con $w \geq \lg S$. Cada operación toca $O(1)$ celdas y las palabras sirven como punteros: se puede acceder a cualquier celda.

Word RAM

Una transdichotomous RAM con las operaciones de un lenguaje como C, todas en $O(1)$:

$$+,\ -,\ \times,\ \div,\ \%,\ \&,\ |,\ \oplus,\ \sim,\ \ll,\ \gg,\ <,\ >$$

Es el modelo de esta unidad: el que mejor describe un computador real.

Cell probe, pointer machine y BST

Cell probe

Solo se cuentan las lecturas y escrituras de celdas; el cómputo es gratis. Sirve para demostrar **cotas inferiores**.

Pointer machine y BST

Pointer machine: solo se siguen punteros. BST: solo se comparan llaves.

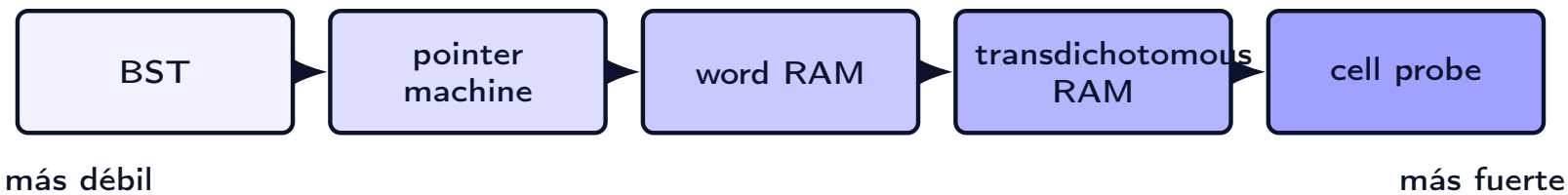
Cell probe, pointer machine y BST

Cell probe

Solo se cuentan las lecturas y escrituras de celdas; el cómputo es gratis. Sirve para demostrar **cotas inferiores**.

Pointer machine y BST

Pointer machine: solo se siguen punteros. BST: solo se comparan llaves.



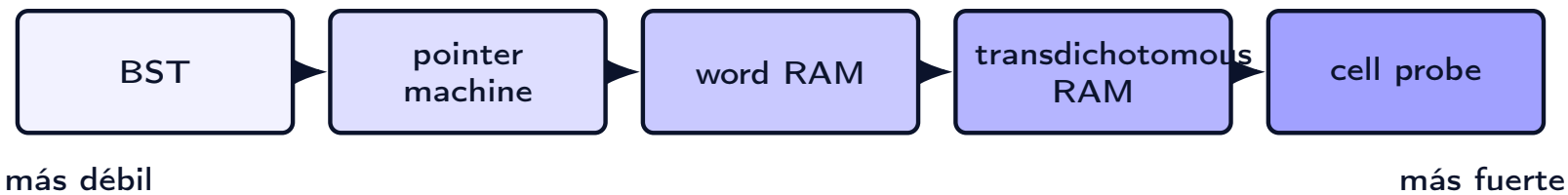
Cell probe, pointer machine y BST

Cell probe

Solo se cuentan las lecturas y escrituras de celdas; el cómputo es gratis. Sirve para demostrar **cotas inferiores**.

Pointer machine y BST

Pointer machine: solo se siguen punteros. BST: solo se comparan llaves.



¿Para qué ordenar los modelos?

Una cota superior en un modelo débil vale en los fuertes, y una cota inferior en un modelo fuerte vale en los débiles: por eso se prueban en cell probe y se diseñan en word RAM.

El problema del predecesor

Predecesor y sucesor

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Mantener un conjunto $S \subseteq U$ de n palabras

- Insertar(x) y Eliminar(x).
- Predecesor(x) $= \max\{y \in S \mid y < x\}$.
- Sucesor(x) $= \min\{y \in S \mid y > x\}$.

Predecesor y sucesor

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Mantener un conjunto $S \subseteq U$ de n palabras

- $\textsc{Insertar}(x)$ y $\textsc{Eliminar}(x)$.
- $\textsc{Predecesor}(x) = \max\{y \in S \mid y < x\}$.
- $\textsc{Sucesor}(x) = \min\{y \in S \mid y > x\}$.

x no tiene que estar en S. Es el problema de la tabla de rutas, y el de "cuál es la siguiente llave" en cualquier índice ordenado.

Los dos extremos

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Una tabla con todas las respuestas

Un arreglo de tamaño u con el predecesor de cada $x \in U$: consulta en $O(1)$, pero $\Theta(u)$ de espacio y de tiempo por actualización.

Un árbol de búsqueda balanceado

$\Theta(n)$ de espacio y $\Theta(\lg n)$ por operación, óptimo en el modelo de comparación.

Los dos extremos

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Una tabla con todas las respuestas

Un arreglo de tamaño u con el predecesor de cada $x \in U$: consulta en $O(1)$, pero $\Theta(u)$ de espacio y de tiempo por actualización.

Un árbol de búsqueda balanceado

$\Theta(n)$ de espacio y $\Theta(\lg n)$ por operación, óptimo en el modelo de comparación.

¿Hay algo en el medio, con tiempo mucho menor que $\lg n$?

Los dos extremos

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Una tabla con todas las respuestas

Un arreglo de tamaño u con el predecesor de cada $x \in U$: consulta en $O(1)$, pero $\Theta(u)$ de espacio y de tiempo por actualización.

Un árbol de búsqueda balanceado

$\Theta(n)$ de espacio y $\Theta(\lg n)$ por operación, óptimo en el modelo de comparación.

¿Hay algo en el medio, con tiempo mucho menor que $\lg n$?

Con $n = 10^9$ y $w = 64$: $\lg n \approx 30$, pero $\lg w = 6$ y $\lg \lg u = \lg w = 6$. Hoy vemos una estructura con ese tiempo.

Árboles de van Emde Boas

Partir las palabras a la mitad

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Coordenadas jerárquicas

Un entero x de w bits se parte en sus $w/2$ bits altos y sus $w/2$ bits bajos:

$$x = \langle c, i \rangle, \qquad c = \text{alto}(x), \quad i = \text{bajo}(x), \quad x = c \cdot \sqrt{u} + i.$$

$c = 11_2 \neq 3\ 01_2 = 1$

1	1	0	1

$x = 13 = 3 \cdot 4 + 1$, con $w = 4$, $u = 16$

Partir las palabras a la mitad

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Coordenadas jerárquicas

Un entero x de w bits se parte en sus $w/2$ bits altos y sus $w/2$ bits bajos:

$$x = \langle c, i \rangle, \qquad c = \text{alto}(x), \quad i = \text{bajo}(x), \quad x = c \cdot \sqrt{u} + i.$$

$c = 11_2 \neq 3\ 01_2 = 1$

1	1	0	1

$x = 13 = 3 \cdot 4 + 1$, con $w = 4$, $u = 16$

c dice en qué *cluster* cae x, e i en qué posición dentro

Hay $\sqrt{u}$ clusters, cada uno con universo de tamaño $\sqrt{u}$. Suponemos que w es potencia de 2.

La recurrencia que buscamos

Si cada operación hiciera una sola llamada recursiva sobre un universo de tamaño $\sqrt{u}$, ¿cuánto costaría?

La recurrencia que buscamos

Si cada operación hiciera una sola llamada recursiva sobre un universo de tamaño $\sqrt{u}$, ¿cuánto costaría?

$T(u) = T(\sqrt{u}) + O(1)$

Con $m = \lg u$: $\sqrt{u} = 2^{m/2}$, así que $S(m) = T(2^m)$ cumple $S(m) = S(m/2) + O(1)$, es decir $S(m) = O(\lg m)$.

$T(u) = O(\lg \lg u) = O(\lg w)$

Cada llamada divide por dos la cantidad de bits: tras $\lg w$ llamadas queda una palabra de 1 bit.

La estructura

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Una vEB V de tamaño u guarda

- $\sqrt{u}$ vEB de tamaño $\sqrt{u}$: $V.cluster[0], \ldots, V.cluster[\sqrt{u}-1]$.
- una vEB $V.summary$ de tamaño $\sqrt{u}$ con los índices c de los clusters **no vacíos**.
- $V.min$, el menor elemento de V (o nulo).
- $V.max$, el mayor elemento de V (o nulo).

La estructura

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Una vEB V de tamaño u guarda

- $\sqrt{u}$ vEB de tamaño $\sqrt{u}$: $V.cluster[0], \ldots, V.cluster[\sqrt{u}-1]$.
- una vEB $V.summary$ de tamaño $\sqrt{u}$ con los índices c de los clusters **no vacíos**.
- $V.min$, el menor elemento de V (o nulo).
- $V.max$, el mayor elemento de V (o nulo).

¿Qué sale caro de esta definición?

La estructura

Una vEB V de tamaño u guarda

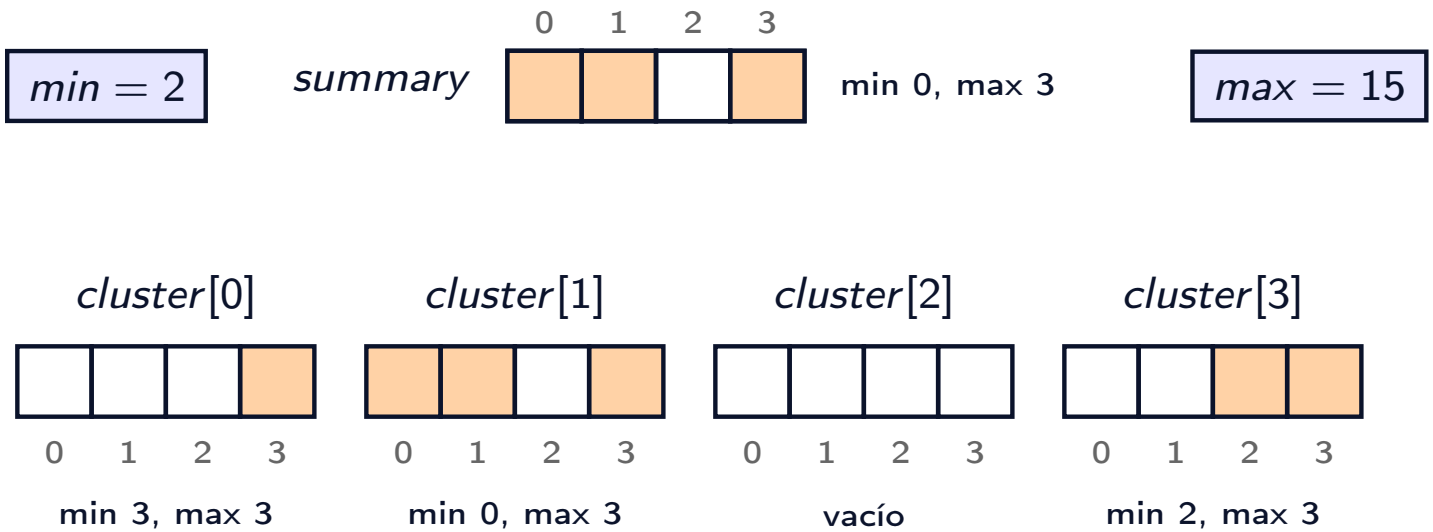
- $\sqrt{u}$ vEB de tamaño $\sqrt{u}$: $V.cluster[0], \ldots, V.cluster[\sqrt{u}-1]$.
- una vEB $V.summary$ de tamaño $\sqrt{u}$ con los índices c de los clusters **no vacíos**.
- $V.min$, el menor elemento de V (o nulo).
- $V.max$, el mayor elemento de V (o nulo).

¿Qué sale caro de esta definición?

Hay una vEB por cada uno de los $\sqrt{u}+1$ componentes

Cada una repite la misma construcción con $u' = \sqrt{u}$ hasta llegar a $u = 2$, donde solo quedan *min* y *max*.

Un ejemplo: $u = 16$ y $S = \{2, 3, 4, 5, 7, 14, 15\}$



Los clusters tienen tamaño 4. El mínimo 2 **no** aparece en ningún cluster: se guarda solo en $V.min$. El resto sí: $3 = \langle 0, 3 \rangle$, $4 = \langle 1, 0 \rangle$, $5 = \langle 1, 1 \rangle$, $7 = \langle 1, 3 \rangle$, $14 = \langle 3, 2 \rangle$, $15 = \langle 3, 3 \rangle$.

Sucesor: un primer intento

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Para SUCESOR($x = \langle c, i \rangle$)

1. Buscar el sucesor de i dentro de $V.cluster[c]$.
2. Si no hay, buscar el siguiente cluster no vacío c': el sucesor de c en $V.summary$.
3. Devolver el mínimo de $V.cluster[c']$.

¿Cuántas llamadas recursivas hace? ¿Qué recurrencia sale?

Sucesor: un primer intento

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Para SUCESOR($x = \langle c, i \rangle$)

1. Buscar el sucesor de i dentro de $V.cluster[c]$.
2. Si no hay, buscar el siguiente cluster no vacío c': el sucesor de c en $V.summary$.
3. Devolver el mínimo de $V.cluster[c']$.

¿Cuántas llamadas recursivas hace? ¿Qué recurrencia sale?

Dos: $T(u) = 2T(\sqrt{u}) + O(1)$, que da $O(\lg u) = O(w)$

Con $m = \lg u$: $S(m) = 2S(m/2) + O(1) = O(m)$. No mejora nada respecto de mirar los w bits uno por uno.

Sucesor: una sola llamada

¿Cómo saber, sin recursión, si el sucesor de x está en su propio cluster?

Sucesor: una sola llamada

¿Cómo saber, sin recursión, si el sucesor de x está en su propio cluster?

Guardando el máximo de cada cluster

El sucesor de i existe dentro de $V.cluster[c]$ exactamente cuando $i < V.cluster[c].max$. Eso se decide en $O(1)$.

Entonces solo se recorre una de las dos ramas

Si $i < V.cluster[c].max$: recursión en el cluster. Si no: recursión en $V.summary$, y el mínimo de ese cluster se lee en $O(1)$.

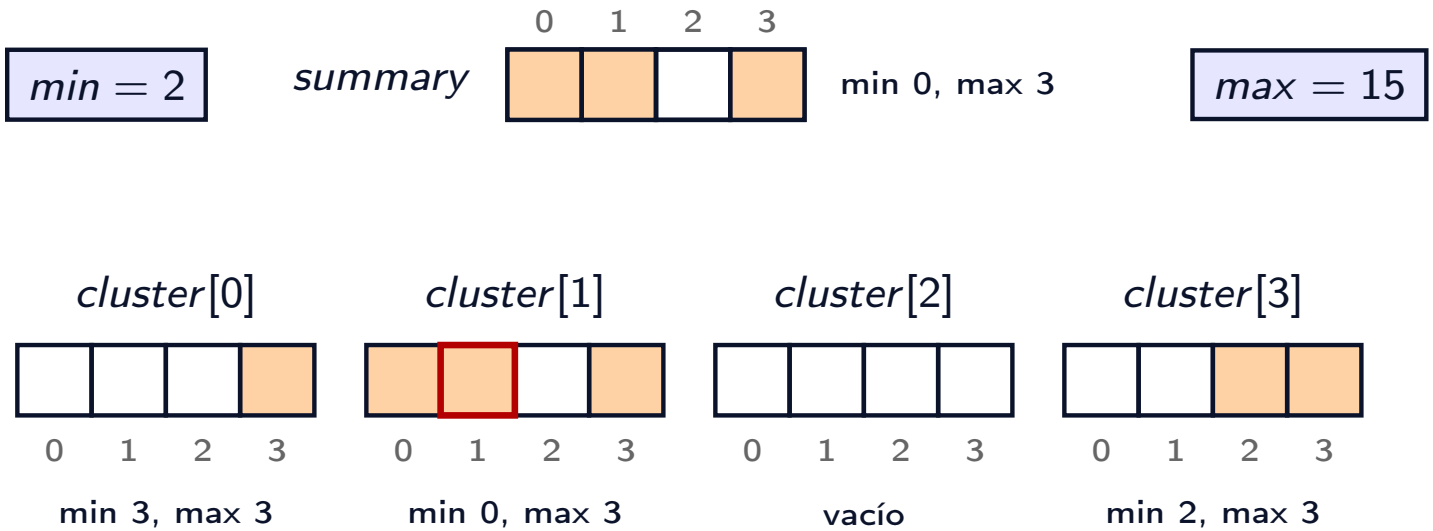
Sucesor

Algoritmo 1: $\textsc{Sucesor}(V, x)$ — menor elemento de V mayor que x

si $V.min \neq nulo$ **y** $x < V.min$ **entonces devolver** $V.min$
si $V.max = nulo$ **o** $x \geq V.max$ **entonces devolver** nulo
si $u = 2$ **entonces devolver** $V.max$
$\langle c, i \rangle \leftarrow x$
si $V.cluster[c].max \neq nulo$ **y** $i < V.cluster[c].max$ **entonces**
  devolver $\langle c,\ \textsc{Sucesor}(V.cluster[c], i) \rangle$
en otro caso
  $c' \leftarrow \textsc{Sucesor}(V.summary, c)$
  devolver $\langle c',\ V.cluster[c'].min \rangle$

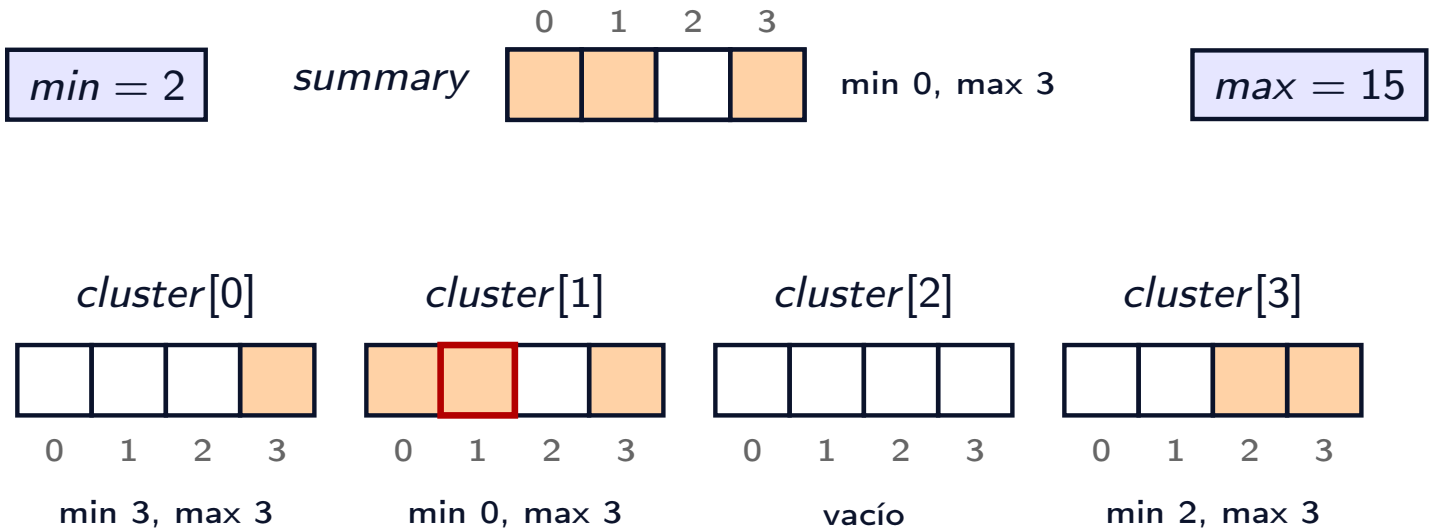
En el caso base ($u = 2$) ya sabemos que $V.min \leq x < V.max$, así que $x = 0$ y el sucesor es $V.max = 1$.

Un ejemplo de sucesor: Sucesor(5)



- $x = 5 = \langle 1, 1 \rangle$: $5 \geq V.min = 2$ y $5 < V.max = 15$.

Un ejemplo de sucesor: Sucesor(5)



- $x = 5 = \langle 1, 1 \rangle$: $5 \geq V.min = 2$ y $5 < V.max = 15$.
- $V.cluster[1].max = 3 > 1$: el sucesor está en el cluster 1. Se busca el sucesor de $i = 1$ en **ese** cluster.

Un ejemplo de sucesor: Sucesor(5)

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Dentro del cluster 1: $\{0, 1, 3\}$ con $u = 4$

Esta vEB tiene $min = 0$ (que no está en sus subclusters), $max = 3$, dos subclusters de tamaño 2 y su propio resumen:

$$\text{subcluster } 0 = \{1\}, \qquad \text{subcluster } 1 = \{3\}, \qquad \text{resumen} = \{0, 1\}.$$

- $i = 1 = \langle 0, 1 \rangle$: el max del subcluster 0 es 1, y $1 < 1$ es falso. Se pregunta al resumen por el sucesor de 0: es 1.

Un ejemplo de sucesor: Sucesor(5)

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

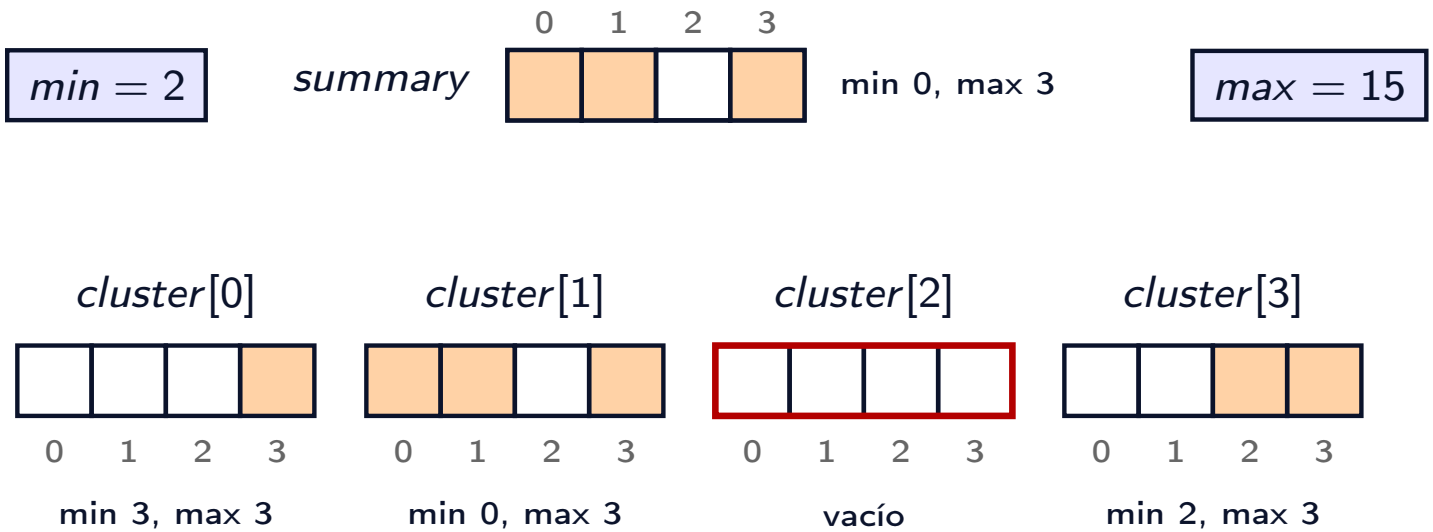
Dentro del cluster 1: $\{0, 1, 3\}$ con $u = 4$

Esta vEB tiene $min = 0$ (que no está en sus subclusters), $max = 3$, dos subclusters de tamaño 2 y su propio resumen:

$$\text{subcluster } 0 = \{1\}, \qquad \text{subcluster } 1 = \{3\}, \qquad \text{resumen} = \{0, 1\}.$$

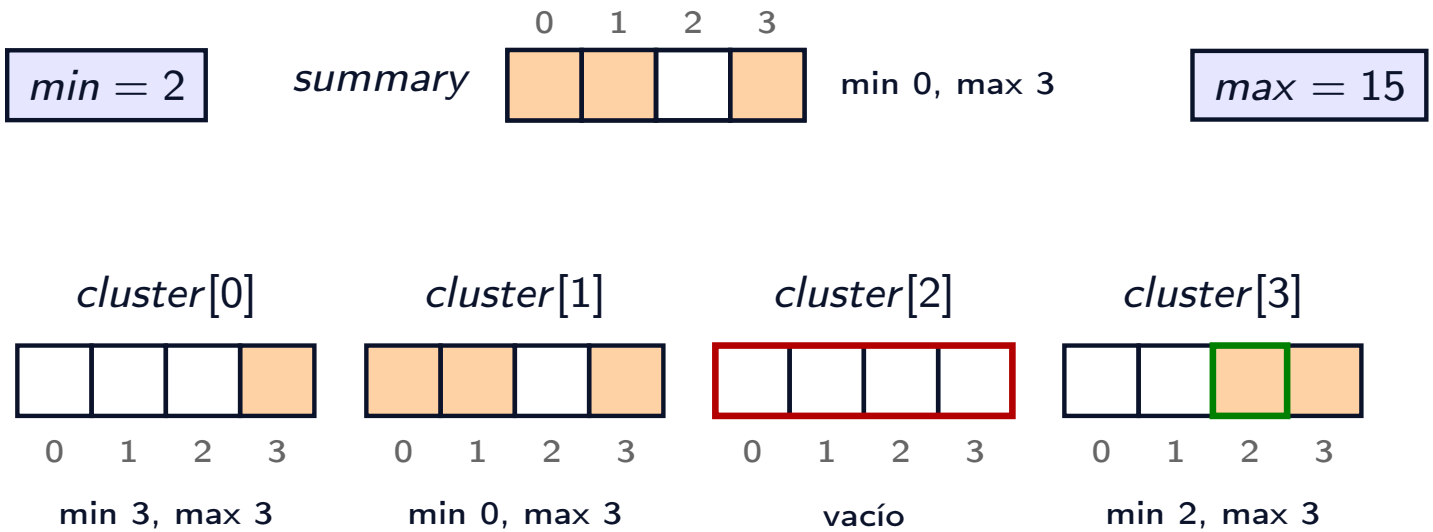
- $i = 1 = \langle 0, 1 \rangle$: el *max* del subcluster 0 es 1, y $1 < 1$ es falso. Se pregunta al resumen por el sucesor de 0: es 1.
- Se devuelve $\langle 1,$ *min* del subcluster $1 \rangle = \langle 1, 1 \rangle = 3$. Afuera, $\langle 1, 3 \rangle = 1 \cdot 4 + 3 = 7$: el sucesor de 5 es 7.

Un ejemplo de sucesor: Sucesor(9)



- $9 = \langle 2, 1 \rangle$. El cluster 2 está vacío: no hay *max* contra el que comparar. Recursión en el resumen: el sucesor de $c = 2$ es $c' = 3$.

Un ejemplo de sucesor: Sucesor(9)



- $9 = \langle 2, 1 \rangle$. El cluster 2 está vacío: no hay *max* contra el que comparar. Recursión en el resumen: el sucesor de $c = 2$ es $c' = 3$.
- Se devuelve $\langle 3,\ V.cluster[3].min \rangle = \langle 3, 2 \rangle = 14$.

Predecesor

Algoritmo 2: PREDECESOR(V, x) — mayor elemento de V menor que x

si $V.max \neq nulo$ **y** $x > V.max$ **entonces devolver** $V.max$
si $V.min = nulo$ **o** $x \leq V.min$ **entonces devolver** nulo
si $u = 2$ **entonces devolver** $V.min$
$\langle c, i \rangle \leftarrow x$
si $V.cluster[c].min \neq nulo$ **y** $i > V.cluster[c].min$ **entonces**
  devolver $\langle c,$ PREDECESOR$(V.cluster[c], i)\rangle$
en otro caso
  $c' \leftarrow$ PREDECESOR$(V.summary, c)$
  si $c' = nulo$ **entonces devolver** $V.min$
  devolver $\langle c',\ V.cluster[c'].max\rangle$

Predecesor

Algoritmo 3: PREDECESOR(V, x) — mayor elemento de V menor que x

si $V.max \neq nulo$ **y** $x > V.max$ **entonces devolver** $V.max$
si $V.min = nulo$ **o** $x \leq V.min$ **entonces devolver** nulo
si $u = 2$ **entonces devolver** $V.min$
$\langle c, i \rangle \leftarrow x$
si $V.cluster[c].min \neq nulo$ **y** $i > V.cluster[c].min$ **entonces**
  devolver $\langle c,\ \text{PREDECESOR}(V.cluster[c], i) \rangle$
en otro caso
  $c' \leftarrow \text{PREDECESOR}(V.summary, c)$
  si $c' = nulo$ **entonces devolver** $V.min$
  devolver $\langle c',\ V.cluster[c'].max \rangle$

¿Por qué "si $c' =$ nulo, devolver $V.min$"? Porque el mínimo no está en ningún cluster. Ej.: PREDECESOR(3): el cluster 0 tiene $min = 3$ y $3 > 3$ es falso; el resumen no tiene $c' < 0$; la respuesta es $V.min = 2$.

Insertar: dos problemas

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Para INSERTAR($x = \langle c, i \rangle$)

Insertar i en $V.cluster[c]$ y, si estaba vacío, también c en $V.summary$.

¿Cuántas llamadas recursivas hay si el cluster estaba vacío?

Insertar: dos problemas

Para INSERTAR($x = \langle c, i \rangle$)

Insertar i en $V.cluster[c]$ y, si estaba vacío, también c en $V.summary$.

¿Cuántas llamadas recursivas hay si el cluster estaba vacío?

Dos, pero una es trivial

Insertar en una vEB **vacía** solo asigna *min* y *max*: $O(1)$. Queda una sola llamada costosa.

Insertar: dos problemas

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Para INSERTAR($x = \langle c, i \rangle$)

Insertar i en $V.cluster[c]$ y, si estaba vacío, también c en $V.summary$.

¿Cuántas llamadas recursivas hay si el cluster estaba vacío?

Dos, pero una es trivial

Insertar en una vEB **vacía** solo asigna *min* y *max*: $O(1)$. Queda una sola llamada costosa.

El mínimo no se guarda recursivamente

Si llega $x < V.min$, se intercambian y el mínimo anterior baja a los clusters. Así el primer elemento de un cluster es su *min*.

Insertar

Algoritmo 4: INSERTAR(V, x)

si $V.min = nulo$ **entonces** $V.min \leftarrow V.max \leftarrow x$, **devolver**
si $x = V.min$ ***o*** $x = V.max$ **entonces devolver**
si $x < V.min$ **entonces** intercambiar $x \leftrightarrow V.min$
si $u > 2$ **entonces**
    $\langle c, i \rangle \leftarrow x$
    si $V.cluster[c].min = nulo$ **entonces**
        INSERTAR($V.summary, c$)
        $V.cluster[c].min \leftarrow V.cluster[c].max \leftarrow i$ `// O(1): estaba vacío`
    en otro caso
        INSERTAR($V.cluster[c], i$)
si $x > V.max$ **entonces** $V.max \leftarrow x$

Un ejemplo de inserción

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Insertar 9 en $S = \{2, 3, 4, 5, 7, 14, 15\}$

$9 = \langle 2, 1 \rangle$ y $9 > V.min = 2$: no hay intercambio. El cluster 2 está vacío: INSERTAR($V.summary, 2$) pasa el resumen a $\{0, 1, 2, 3\}$ (aquí sí hay recursión) y $V.cluster[2].min \leftarrow V.cluster[2].max \leftarrow 1$, sin recursión. Como $9 \leq V.max = 15$, no hay más que hacer.

Un ejemplo de inserción

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Insertar 9 en $S = \{2, 3, 4, 5, 7, 14, 15\}$

$9 = \langle 2, 1 \rangle$ y $9 > V.min = 2$: no hay intercambio. El cluster 2 está vacío: INSERTAR($V.summary, 2$) pasa el resumen a $\{0, 1, 2, 3\}$ (aquí sí hay recursión) y $V.cluster[2].min \leftarrow V.cluster[2].max \leftarrow 1$, sin recursión. Como $9 \leq V.max = 15$, no hay más que hacer.

Insertar 0 a continuación

$0 < V.min = 2$: se intercambian, $V.min = 0$ y ahora se inserta el 2 viejo: $\langle 0, 2 \rangle$ entra al cluster 0, que ya tiene $\{3\}$, con una llamada recursiva.

Eliminar

Si borramos $V.min$, ¿cuál es el nuevo mínimo, y dónde estaba guardado?

Eliminar

Si borramos $V.min$, ¿cuál es el nuevo mínimo, y dónde estaba guardado?

Es el mínimo del primer cluster no vacío, y estaba en ese cluster

Con $c = V.summary.min$, el nuevo mínimo es $\langle c,\ V.cluster[c].min \rangle$. Pasa a ser $V.min$ y se **elimina de su cluster**, para no repetirlo.

Después, como en Insertar, una sola llamada costosa

Se elimina i de $V.cluster[c]$. Si el cluster queda vacío, esa eliminación fue $O(1)$ (era un solo elemento) y entonces se elimina c del resumen. Falta recalcular $V.max$.

Eliminar

Algoritmo 5: ELIMINAR(V, x) — con $x \in V$

si $V.min = V.max$ **entonces** $V.min \leftarrow V.max \leftarrow$ nulo, **devolver**
si $u = 2$ **entonces** $V.min \leftarrow V.max \leftarrow 1 - x$, **devolver**
si $x = V.min$ **entonces**
  $c \leftarrow V.summary.min$, $x \leftarrow \langle c,\ V.cluster[c].min \rangle$, $V.min \leftarrow x$
$\langle c, i \rangle \leftarrow x$, ELIMINAR$(V.cluster[c], i)$
si $V.cluster[c].min = nulo$ **entonces**
  ELIMINAR$(V.summary, c)$
  si $x = V.max$ **entonces**
    si $V.summary.min = nulo$ **entonces**
      $V.max \leftarrow V.min$
    en otro caso
      $c' \leftarrow V.summary.max$, $V.max \leftarrow \langle c',\ V.cluster[c'].max \rangle$
en otro caso
  si $x = V.max$ **entonces** $V.max \leftarrow \langle c,\ V.cluster[c].max \rangle$

Un ejemplo de eliminación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Eliminar 2 (el mínimo) de $S = \{2, 3, 4, 5, 7, 14, 15\}$

$c = V.summary.min = 0$ y $V.cluster[0].min = 3$: el nuevo mínimo es $\langle 0, 3 \rangle = 3$.

Un ejemplo de eliminación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Eliminar 2 (el mínimo) de $S = \{2, 3, 4, 5, 7, 14, 15\}$

$c = V.summary.min = 0$ y $V.cluster[0].min = 3$: el nuevo mínimo es $\langle 0, 3 \rangle = 3$.

- Hay que sacar el 3 de su cluster. Ese cluster tiene un solo elemento, así que su *min* y *max* pasan a nulo en $O(1)$.
- El cluster quedó vacío: $\textsc{Eliminar}(V.summary, 0)$, y el resumen pasa de $\{0, 1, 3\}$ a $\{1, 3\}$. Como $x = 3 \neq V.max = 15$, no hay que tocar $V.max$.

Un ejemplo de eliminación

Eliminar 2 (el mínimo) de $S = \{2, 3, 4, 5, 7, 14, 15\}$

$c = V.summary.min = 0$ y $V.cluster[0].min = 3$: el nuevo mínimo es $\langle 0, 3 \rangle = 3$.

- Hay que sacar el 3 de su cluster. Ese cluster tiene un solo elemento, así que su *min* y *max* pasan a nulo en $O(1)$.
- El cluster quedó vacío: $\textsc{Eliminar}(V.summary, 0)$, y el resumen pasa de $\{0, 1, 3\}$ a $\{1, 3\}$. Como $x = 3 \neq V.max = 15$, no hay que tocar $V.max$.

Resultado

$S = \{3, 4, 5, 7, 14, 15\}$, con $V.min = 3$ y ningún elemento en el cluster 0.

Cuánto cuestan las operaciones

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Teorema

SUCESOR, PREDECESOR, INSERTAR y ELIMINAR cuestan $O(\lg \lg u) = O(\lg w)$.

Cuánto cuestan las operaciones

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Teorema

SUCESOR, PREDECESOR, INSERTAR y ELIMINAR cuestan $O(\lg \lg u) = O(\lg w)$.

En cada llamada, a lo más una subllamada no trivial

- SUCESOR/PREDECESOR: o el cluster o el resumen, nunca ambos.
- INSERTAR: si el cluster está vacío, esa inserción es $O(1)$ y queda solo el resumen; si no, solo el cluster.
- ELIMINAR: si el cluster queda vacío, su eliminación fue $O(1)$ (un solo elemento) y queda solo el resumen.

Entonces $T(u) \leq T(\sqrt{u}) + O(1)$, que da $O(\lg \lg u)$.

¿Y el espacio?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¿Cuánta memoria ocupa una vEB de tamaño u?

¿Y el espacio?

¿Cuánta memoria ocupa una vEB de tamaño u?

$S(u) = (\sqrt{u} + 1)\, S(\sqrt{u}) + O(1)$

Hay $\sqrt{u}$ clusters y un resumen, todos de tamaño $\sqrt{u}$, y se reservan aunque estén vacíos. Si $S(2) = a$ y el término constante es a, por inducción $S(u) \leq 2a\,u - 3a$: $(\sqrt{u} + 1)(2a\sqrt{u} - 3a) + a = 2a\,u - a\sqrt{u} - 2a \leq 2a\,u - 3a$.

¿Y el espacio?

¿Cuánta memoria ocupa una vEB de tamaño u?

$S(u) = (\sqrt{u}+1)\,S(\sqrt{u}) + O(1)$

Hay $\sqrt{u}$ clusters y un resumen, todos de tamaño $\sqrt{u}$, y se reservan aunque estén vacíos. Si $S(2) = a$ y el término constante es a, por inducción $S(u) \leq 2a\,u - 3a$: $(\sqrt{u}+1)(2a\sqrt{u} - 3a) + a = 2a\,u - a\sqrt{u} - 2a \leq 2a\,u - 3a$.

$S(u) = \Theta(u)$

Con $w = 32$: $u \approx 4 \cdot 10^9$ palabras, aunque S tenga 10 elementos. Con $w = 64$ ni siquiera cabe en memoria.

¿Y el espacio?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¿Se puede tener el mismo tiempo $O(\lg w)$ con $\Theta(n)$ de espacio?

¿Y el espacio?

¿Se puede tener el mismo tiempo $O(\lg w)$ con $\Theta(n)$ de espacio?

Es el tema de la próxima clase

La memoria de una vEB crece con el universo, no con n.

Y en la pointer machine, $O(\lg \lg u)$ por operación es lo mejor posible, también con $\Omega(u)$ de espacio.

Ordenar con enteros

Ordenar con una vEB

Algoritmo 6: ORDENAR-VEB($A[1..n]$)

$V \leftarrow$ vEB vacía de tamaño u
para $j \leftarrow 1$ **a** n **hacer** INSERTAR($V, A[j]$)
$x \leftarrow V.min$
mientras $x \neq nulo$ **hacer**
  imprimir x, $x \leftarrow$ SUCESOR(V, x)

Ordenar con una vEB

Algoritmo 7: ORDENAR-VEB($A[1..n]$)

$V \leftarrow$ vEB vacía de tamaño u
para $j \leftarrow 1$ **a** n **hacer** INSERTAR($V, A[j]$)
$x \leftarrow V.min$
mientras $x \neq nulo$ **hacer**
  imprimir x, $x \leftarrow$ SUCESOR(V, x)

$O(n \lg w)$, con $\Theta(u)$ de espacio

n inserciones y n sucesores. Con $w = 64$, $\lg w = 6$: contra $\lg n \approx 30$ de un ordenamiento por comparaciones.

Ordenar con una vEB

Algoritmo 8: ORDENAR-VEB($A[1..n]$)

$V \leftarrow$ vEB vacía de tamaño u
para $j \leftarrow 1$ **a** n **hacer** INSERTAR($V, A[j]$)
$x \leftarrow V.min$
mientras $x \neq nulo$ **hacer**
  imprimir x, $x \leftarrow$ SUCESOR(V, x)

$O(n \lg w)$, con $\Theta(u)$ de espacio

n inserciones y n sucesores. Con $w = 64$, $\lg w = 6$: contra $\lg n \approx 30$ de un ordenamiento por comparaciones.

Otros ordenamientos de enteros

Counting sort: $O(n + u)$. Radix sort con dígitos de $\lg n$ bits: $O(n\,w/\lg n)$, lineal si $w = O(\lg n)$. La vEB cubre $w = \lg^{O(1)} n$ con $O(n \lg \lg n)$.

Ordenar y cola de prioridad son lo mismo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

De cola de prioridad a ordenar

Con una cola de prioridad que haga INSERTAR y EXTRAER-MÍNIMO en $O(f(n, w))$: insertar los n elementos y extraerlos uno a uno ordena en $O(n\, f(n, w))$.

Ordenar y cola de prioridad son lo mismo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

De cola de prioridad a ordenar

Con una cola de prioridad que haga INSERTAR y EXTRAER-MÍNIMO en $O(f(n, w))$: insertar los n elementos y extraerlos uno a uno ordena en $O(n\, f(n, w))$.

De ordenar a cola de prioridad (Thorup)

Si se pueden ordenar n enteros de w bits en $O(n\, S(n, w))$, existe una cola de prioridad con INSERTAR, ELIMINAR y MÍNIMO en $O(S(n, w))$.

Una cola de prioridad de tiempo constante exigiría ordenar en $O(n)$

Es un problema abierto para w arbitrario.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Las estructuras de enteros viven en la word RAM: palabras de w bits y operaciones de C en $O(1)$. Las cotas inferiores se prueban en cell probe.
- El problema del predecesor admite $O(\lg w)$, mucho menos que $\lg n$, porque las llaves son palabras y no cajas negras.
- vEB: se parten los bits en mitades, se guarda *min* aparte y *max* en cada nivel, y así toda operación hace una sola llamada costosa: $T(u) = T(\sqrt{u}) + O(1)$.
- El espacio es $\Theta(u)$. Ordenar con vEB cuesta $O(n \lg w)$, y ordenar y cola de prioridad son equivalentes.

Logros de la sesión



Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos distinguir word RAM, cell probe y pointer machine, y qué cotas se transfieren entre ellos
- Sabemos plantear el problema del predecesor y sus extremos triviales

Logros de la sesión

- Sabemos distinguir word RAM, cell probe y pointer machine, y qué cotas se transfieren entre ellos
- Sabemos plantear el problema del predecesor y sus extremos triviales
- Sabemos implementar y analizar las cuatro operaciones de una vEB, y por qué cada una hace una sola llamada costosa
- Sabemos ordenar con una vEB y relacionarlo con colas de prioridad

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias

Referencias

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Bibliografía citada

- van Emde Boas, P. (1975). Preserving order in a forest in less than logarithmic time. *Proceedings of FOCS'75*, 75–84.
- Thorup, M. (2007). Equivalence between priority queues and sorting. *Journal of the ACM*, 54(6).
- Demaine, Erik (2012). *6.851: Advanced Data Structures (Spring'12)*, Lecture 11. `https://courses.csail.mit.edu/6.851/spring12/`.

Agradecimientos

Estas diapositivas están basadas en las clases de Luciano A. Romero Calla para este curso (a su vez basadas en las clases de Erik Demaine, MIT 6.851), así como en clases similares de otras universidades.
Lista de créditos: Luciano A. Romero Calla, Erik Demaine, Keith Schwarz.